

Git and GitHub



Prepared by
Eskeidar Y.



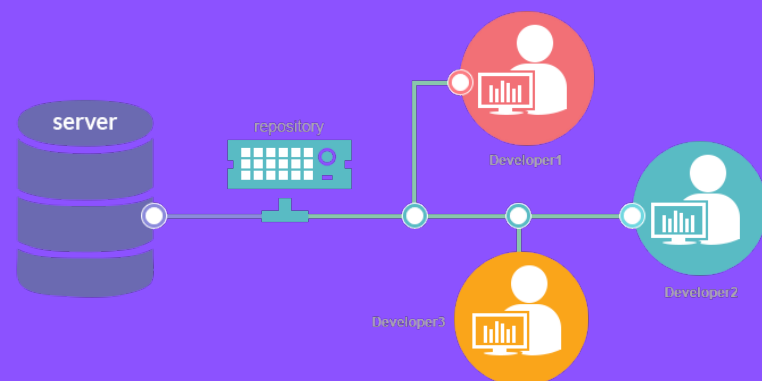
Outline

- What Git and Github
- Project setup
- Git commands
 - Get Initialization
 - Git global configuration
- GitHub account creation
- Create repository in github
- Push local project in to github repository
- Clone project from github repository



What is Git?

- Git is a popular version control system. It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then.
- It is used for:
 - **Tracking code changes:** monitoring and recording modifications made to a piece of code
 - **Tracking who made changes**
 - **Coding collaboration** : Multiple developers can work on the same code





What does Git do?

- Manage projects with Repositories
- Clone a project to work on a local copy
- Control and track changes with Staging and Committing
- Branch and Merge to allow for work on different parts and versions of a project
- Pull the latest version of the project to a local copy
- Push local updates to the main project



Git Installation

- Download it from <https://git-scm.com/downloads> and install it
- Check git version `git --version`



Project setup

Crate folder

Basic

- **Working directory:** Where you edit files.
- **Staging area:** Where you prepare specific changes for a commit.
- **Commit:** A saved snapshot of your staged changes.



commands



Git global configuration

```
git config --global user.name " your user name"
```

```
git config --global user.email " your email"
```

- **To Change branch name**

```
git config --global init.default branch "main"
```

- **To view global configuration**

```
git config -global--list
```




Get Initialization

- git init
 - This command creates an empty Git repository. It sets up git. Git will start tracking all changes in the current folder
- Git status
 - Show all changes, this command only show name of changed and new added files, Show untracked file.
- Git diff: this command show code changes in chanced folders (if we move file in staging directory it dose not show the change)



Start tracking/ stage changes

Adding from working directory to staging directory

- Git add file/folder

Git add file : -For Single file Git add index.html

Git add folder/

Pick all files in folder command line is running In

Git add .

get add --all

Git add -a



Commit the change

- Version in git is known as a **commit**
- Register on git history (from staging to commit)
`git commit -m "important comment"`
- If we miss something
`Git add .`
`Git commit -a -m "comment" - amend`
- To explore the history and tracking changes in your codebase
`Git log`
- Short hand form all steps in single line
`Git commit -a -m "comment"`



View git project change

- Show the file that are being tracked
Git ls -files
- To stop tracking file
Git rm -cached **filename**
- To restore
Git restore -staged **filename**
- Show change in staging
Git diff -cached
Git diff -stage
- To remove from stging directory
Git restore --staged **filename**



Git History

- To View commit history
Git log
Git show
- To View the summary of commit history
Git short log

Git status Git diff	To show change in working directory
Git diff - -Staging	To show change in staging directory
Git log	To show commit history



Git History

Git log command show the following information

- SHA : 40 characters SHA key
- Branch source and destination
- Author name and email
- Comment message in commit

```
PS C:\Users\esku2\OneDrive\Desktop\git practice> git log
commit f61cd69cd577ecb493cd3ba8300dac04cc71770f (HEAD -> main)
Author: Esku <yirgaeskedar9@gmail.com>
Date:   Fri Jan 24 11:51:11 2025 +0300

    version 2 about is added

commit d45b7bfc02793be7fba212ba4e13f4056e10d855
Author: Esku <yirgaeskedar9@gmail.com>
Date:   Fri Jan 24 10:53:04 2025 +0300

    version 1
```



Git History

Git log parameters

git log -3 show 3 latest commit

git log --after/befor="2025-24-1"

git log -- author="name of author"

git log --grep="comment message"

git log -- filename

git log -S "code line"

git log --patch also show code change (git log -P)

git log --patch -2

git log --stat show a summary

git log --oneline show very short summary



Git History

Git log parameters (cont...)

git log --graph show branch hierarchy graphically

git log --pretty=oneline

git log --pretty=format:"format specification"

git log --pretty="%h -%an, %ar : %s"

Format Specifier	Description	Format Specifier	Description
%H	Full commit hash – 40 chars	%ae	Author email
%h	Abbreviated hash – 7 digit	%ad	Author date (format respects the --date=option)
%T	Tree hash	%ar	Author date, relative to current date
%t	Abbreviated tree hash	%cn	Committer name
%P	Parent hashes	%ce	Committer email
%p	Abbreviated Parent hashes	%cd	Committer date
%an	Author name	%cr	Committer date, relative
		%s	Subject (comment)



Git History

- To change **latest commit** comment
git commit -m "new comment" -amend
- To back into previous version
git checkout **SHA**
- To permanently back to previous version



git reset --hard SHA



Git History

git show : similar to git log in addition information's that is view by git log it also show code change in latest commit

git show SHA

git shortlog :it show short summarize commit information it shows:

- Author
- Commit comment



Git branch

What is git branch

- it is a copy/snapshot of git project, it contain all file of a project
- In Git, a branch is a new/separate version of the main repository.
- A Git branch is a pointer to a specific snapshot (commit) in your project's version history.

Why git branch

- **To add new feature**
- **To fix bugs:** fix bugs or experiment without affecting the main codebase.

Main use of git branch

- To protect main branch from unintended changes



Git branch

- To view the current branch
git branch
git branch -a it show all branch current branch marked by asterisk (*)
- To Create a New Branch
git branch <branch-name>
git branch add_CSS
- To switch the new branch
git checkout <branch-name>
git checkout add_CSS



Git branch

- To compare difference in a branch
`git diff add_CSS`
- Create new branch and switch to new bench add in single command
`git checkout -b JavaScript_added` **or**
`Git switch -c JavaScript_added`
- **To include main branch copy plus other branch change**
`Git checkout -b <new branch name> <existed branch>`
`Git checkout -b add_CSSbootS add_CSS`
- To rename current branch name
`Git rename -m newname`
- To rename other branch name
`Git rename -m newname oldname`



Git branch

Git branch name naming rule and conventions

- Descriptive and Clear:
 - The branch name should indicate the purpose or task it represents. Use short, meaningful, and easy-to-understand names
- Use Lowercase Letters:
 - Most teams prefer lowercase letters to avoid confusion (e.g., `feature/login-page` rather than `Feature/LoginPage`).
- Separate Words with Hyphens or Slashes:
 - Use hyphens (-) for readability or slashes (/) to create logical categories. Examples: `feature/add-search`, `fix/login-bug`.



Git branch

Git branch name naming rule and conventions(cont...)

- Avoid Special Characters:
 - Stick to letters, numbers, hyphens (-), dot(.) and slashes (/). Avoid spaces or characters like @, #, *, or \$.
 - Should not start with dot(.)
 - Should not end with slash(/)
- not Allow to use Reserved keyword
 - HEAD
- Good habit
 - Descriptive
 - Short
 - Consistency



Git branch

Git branch name naming rule and conventions(cont...)

Branch name	Purpose	Example
main or prod	Production branch, used to host production codebase	Project/main, Project/prod
dev or test	Development branch, used to host codebase that can be modified and pass through unit tests.	Project/dev Project/test
feature/feature_description	New feature branch	Project/feature/add_qstns Project/feature/add_modal Project/feature/sbmt_qstns
release/release_description	Release branch	Project/release/v1.0.5
hotfix/fix_description	Hotfix branch	Project/hotfix/sbm_qstns
bugfix/bug_description	Debug branch	Project/bugfix/sbm_qstns



Git branch

- To combine new changes to main branch
git checkout <destination branch >
git merge -m "comment" <source branch>
- To combine multiple changes to main branch
git checkout <destination branch >
git merge -m "comment" <source branch1 ><source branch2>

To delete a branch

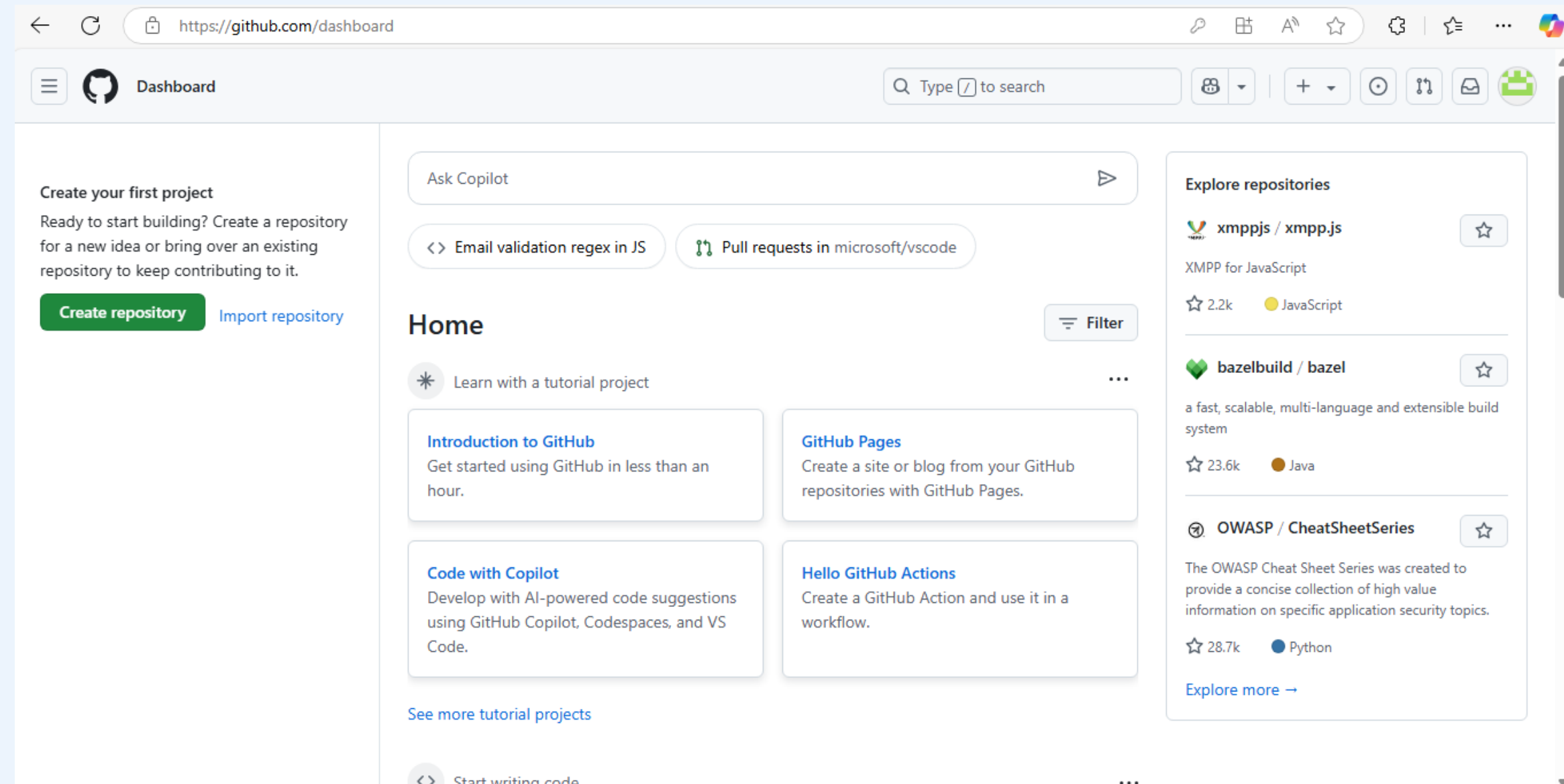
git branch -D < branch name>

[Git & GitHub - Part 1: Introduction \(Amharic Lang.\)](#)



GitHub account creation

- Go to github.com and create GitHub account
- After successfully create GitHub account you will see this dashboard

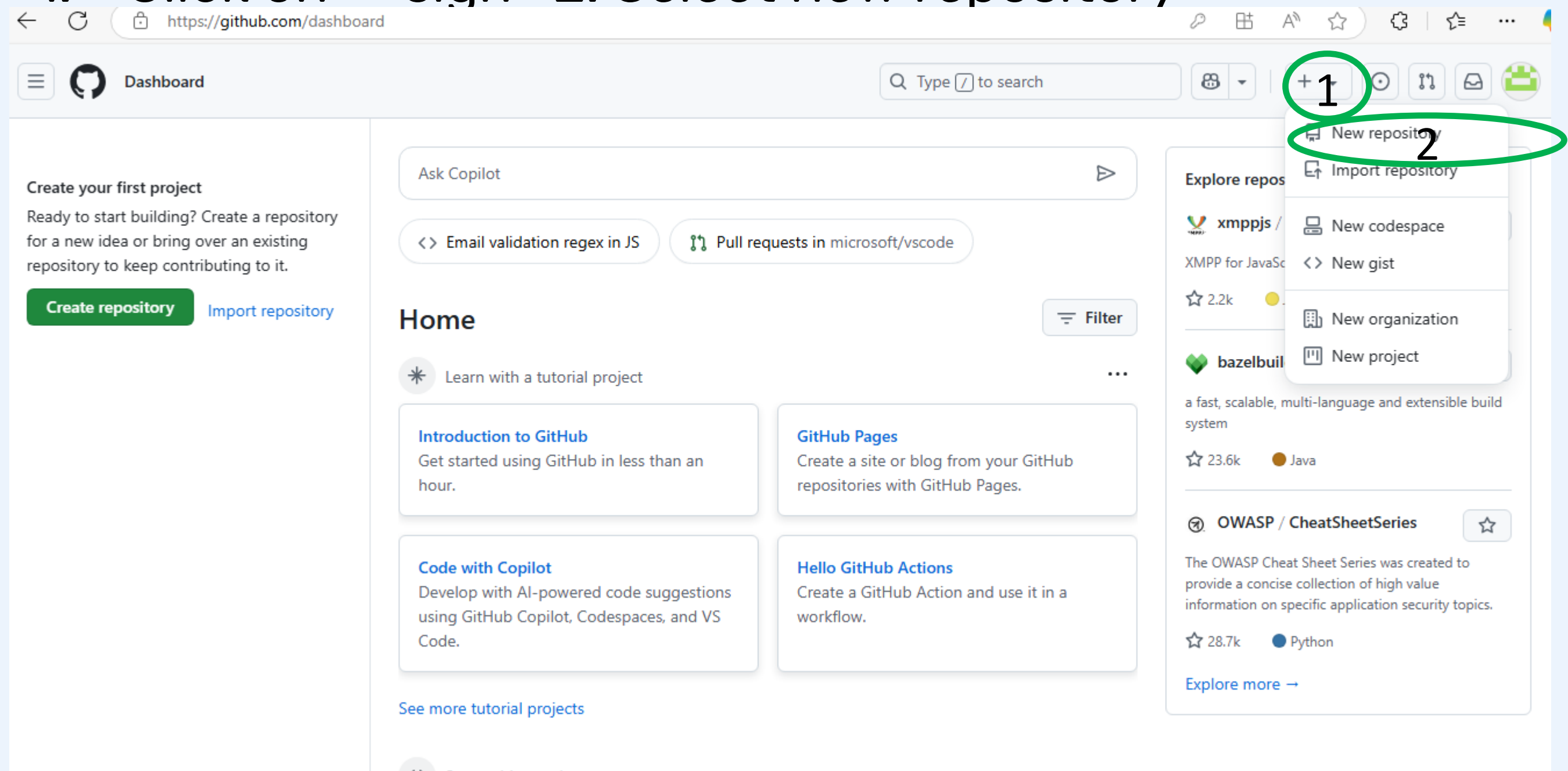




Create repository in githubt

- A GitHub repository is where your project and its version history live. To cerate github repository:

1. Click on + sign 2. Select new repository





Push local project in to github

git branch -M main

git remote add origin <https://github.com/learn-git-basics/Simple-website-to-learn-github.git>

git push -u origin main



Using Github for Team Collaboration

Create new repository

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Owner *

chadMissionBit

Repository name *

team-collaboration

Great repository names are short and memorable. Need inspiration? How about [animated-invention](#)?

Description (optional)

A project used for demonstrating team collaboration.



Public

Anyone on the internet can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☒ Add a README file

This is where you can write a long description for your project. [Learn more](#).

☐ Add .gitignore

Choose which files not to track from a list of templates. [Learn more](#).

☐ Choose a license

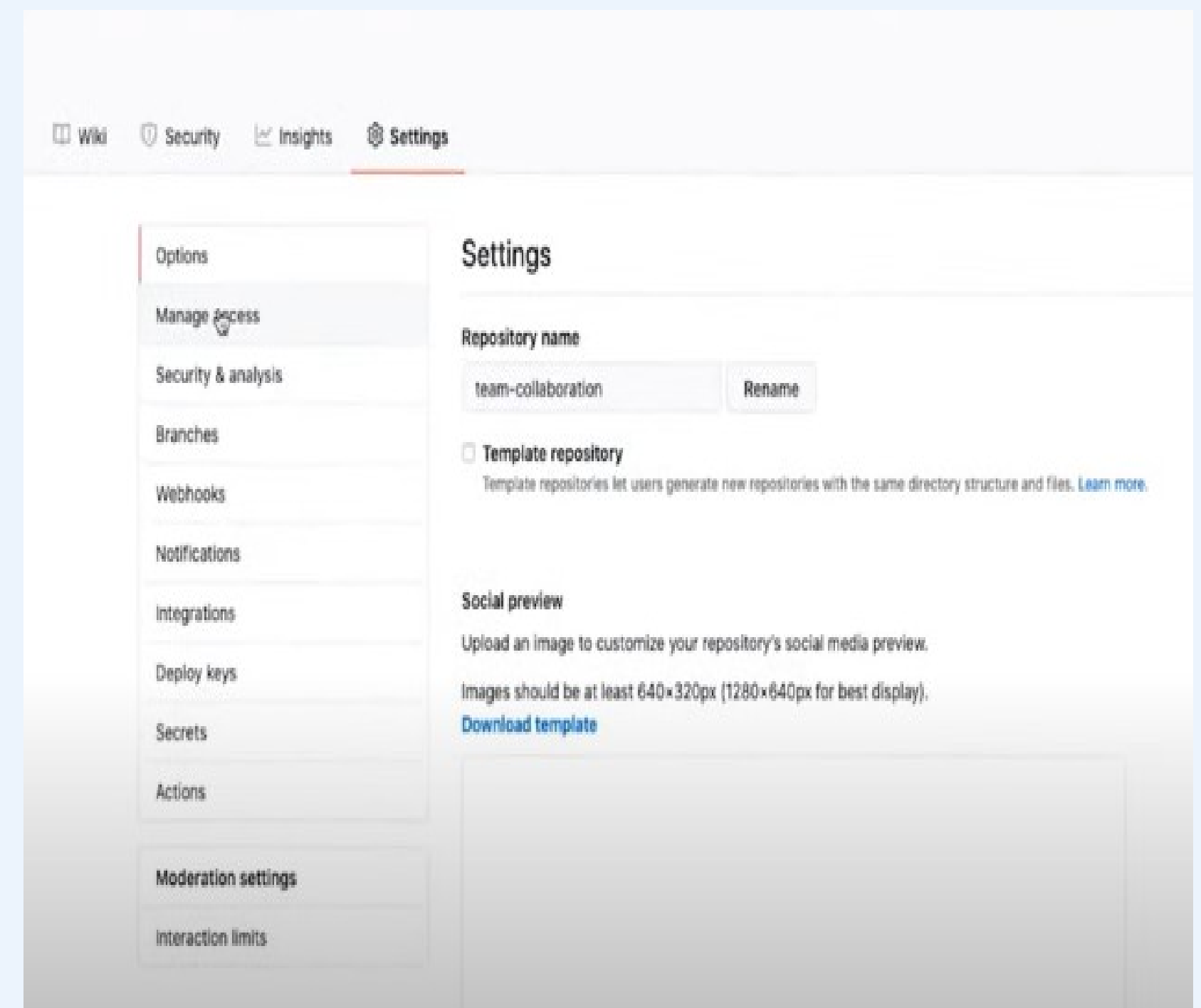
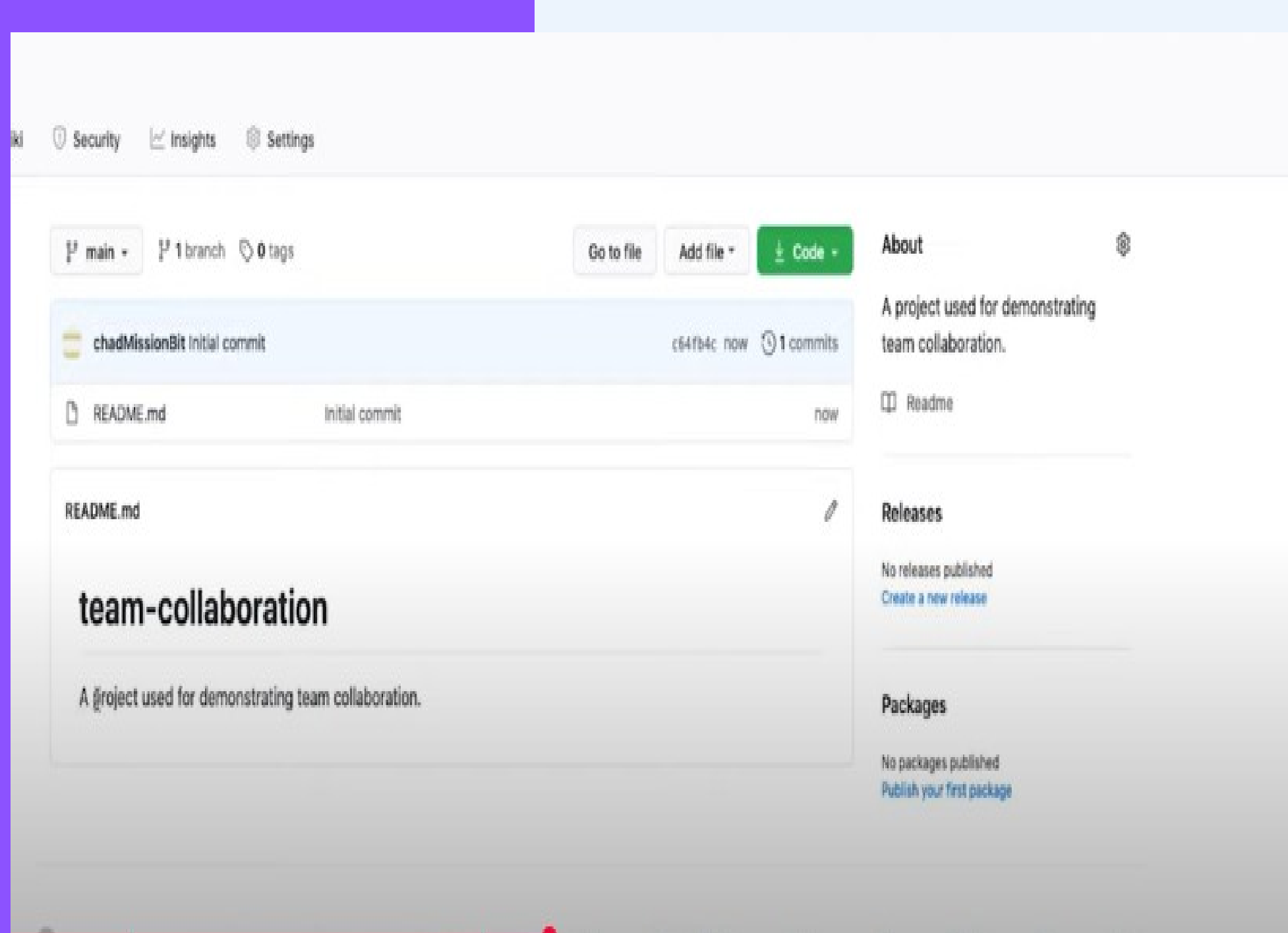
A license tells others what they can and can't do with your code. [Learn more](#).

This will set `main` as the default branch. Change the default name in your [settings](#).



Using Github for Team Collaboration

Goto setting and select manage access





Using Github for Team Collaboration

Invite you team member using github username

